

1. Компилятор GCC (GNU Compiler Collection)

Назначение: Превращает исходный код (C/C++) в исполняемый файл.

Базовые команды:

- `gcc hello.c` → создает `a.out` (исполняемый файл по умолчанию).
- `gcc hello.c -o hello` → создает исполняемый файл с именем `hello`.
- `./hello` → запуск программы из текущей папки (т.к. `./` указывает на текущий каталог).

3 этапа работы GCC:

1. **Препроцессинг (-E):** Обрабатывает директивы `#include` и `#define`, вставляя содержимое заголовочных файлов в код.
2. **Компиляция (-c):** Превращает код в машинные команды. Результат — **объектный файл** (`.o` или `.obj`). Не запускается.
3. **Компоновка (линковка, -o):** Связывает все объектные файлы проекта и подключает библиотеки. Результат — готовый исполняемый файл.

Важное правило: Если программа использует математические функции (`sin`, `cos`, `pow`), нужно явно подключить математическую библиотеку флагом `-lm`.

Пример: `gcc calculate.o main.o -o kalkul -lm`

2. Make-файлы (Makefile)

Назначение: Автоматизация сборки проекта, чтобы не вводить длинные команды вручную.

Структура Makefile:

makefile

цель: зависимости

команда (ОБЯЗАТЕЛЬНО с отступом Tab)

Пример из лабораторной:

makefile

kalkul: calculate.o main.o

gcc calculate.o main.o -o kalkul -lm

calculate.o: calculate.c calculate.h

gcc -c calculate.c

main.o: main.c calculate.h

gcc -c main.c

clean:

```
rm -f kalkul calculate.o main.o
```

Ключевые правила:

- `make` — собрать проект.
 - `make clean` — удалить все собранные файлы (объектные и исполняемый).
 - `make install` — скопировать программу в `/usr/local/bin` (требует прав суперпользователя `su`).
-

3. Процессы в Linux

Контекст процесса (данные, хранящиеся о процессе):

- **Пользовательский контекст:** Код программы, данные (переменные, константы), стек, динамическая память.
- **Контекст ядра:** Системные данные (регистры, стек ядра) и идентификаторы: **PID** (ID процесса), **PPID** (ID родителя), **UID** (ID пользователя).

Иерархия:

- Все процессы создаются другим процессом (кроме процесса `kernel` с PID 0).
 - Процесс-родитель создает процесс-ребенка.
 - Если родитель умирает раньше ребенка, ребенок "усыновляется" процессом `init` (PID 1).
-

4. Системные вызовы (основные функции)

`getpid()` и `getppid()` — возвращают ID текущего процесса и ID его родителя.

```
c
#include <sys/types.h>
#include <unistd.h>
pid_t getpid(void);
pid_t getppid(void);
```

`fork()` — **ЕДИНСТВЕННЫЙ** способ создать новый процесс в Unix.

- Вызывается 1 раз, а возвращается 2 раза (в родителе и в ребенке).
- **Возвращаемое значение:**
 - В **родительский** процесс → возвращается **PID ребенка** (положительное число).
 - В **дочерний** процесс → возвращается **0**.
 - При ошибке → **-1**.

Шаблон использования:

с

```
pid = fork();  
if (pid == -1) { /* Ошибка */ }  
else if (pid == 0) { /* Код для ребенка */ }
```

- `else { /* Код для родителя */ }`

exit(status) — завершает процесс (сбрасывает буферы ввода-вывода).

- `status = 0` — успешное завершение.
- Если родитель не получит статус завершения, процесс становится **зомби** (defunct).

exec() (набор функций: **exec1p**, **execvp**, **exec1**, **execv**, **execle**, **execve**) — заменяет **текущий** процесс новой программой.

- **Важно:** Создает **НОВЫЙ** процесс (в отличие от **fork**).
- **Что сохраняется:** PID, PPID, открытые файлы (если явно не сказано закрыть).
- **Что меняется:** Весь код и данные процесса заменяются на код загружаемой программы.
- При успехе — **не возвращает управление** обратно в вызывающую программу.

Главное отличие:

- **fork()** → создает **клон** текущего процесса (новый процесс).
- **exec()** → заменяет **текущий** процесс на другую программу (новый процесс не создается).